第10章 文件操作

10.1 学习目标

- 理解文件的概念和分类
- 掌握文件的打开与关闭
- 掌握文件的读写操作
- 了解文件定位和随机读写
- 能编写涉及文件操作的完整程序

10.2 文件概述

10.2.1 为什么需要文件

```
// 之前：数据在程序结束时就丢失了
int scores[100];
scanf(...);
printf(...);
// 程序结束 → 数据消失

// 有文件：数据持久化，下次运行还能用
fprintf(fp, ...);   // 写入文件
fscanf(fp, ...);    // 从文件读取
// 数据保存在磁盘上，永久存储
```

10.2.2 文件的分类

分类方式	类型	说明
按存储形式	**文本文件（ASCII）**	以字符形式存储，可读
按存储形式	**二进制文件**	以内存中的二进制形式存储
按访问方式	顺序文件	按顺序读写
按访问方式	随机文件	可跳转到任意位置读写

10.2.3 文件指针 FILE *

```
FILE *fp;   // FILE 是 stdio.h 中定义的结构体类型
            // fp 是指向文件的指针，用于标识一个打开的文件
```

10.3 文件的打开与关闭

10.3.1 fopen —— 打开文件

```
FILE *fopen(const char *filename, const char *mode);
// 成功返回文件指针，失败返回 NULL
```

10.3.2 打开模式

模式	含义	文件不存在	文件已存在
`"r"`	只读	出错	打开
`"w"`	只写	创建新文件	**清空**原内容
`"a"`	追加	创建新文件	在末尾追加
`"r+"`	读写	出错	打开
`"w+"`	读写	创建新文件	**清空**原内容
`"a+"`	读写（追加）	创建新文件	在末尾追加
`"rb"`	二进制读	出错	打开
`"wb"`	二进制写	创建新文件	**清空**原内容

> 注意 `"w"` 模式会**清空文件原有内容**，谨慎使用！

10.3.3 fclose —— 关闭文件

```
int fclose(FILE *fp);
// 成功返回0，失败返回EOF(-1)
```

10.3.4 标准打开关闭流程

```
#include <stdio.h>

int main()
{
    FILE *fp;

    // 打开文件
    fp = fopen("data.txt", "w");
    if (fp == NULL)
    {
        printf("文件打开失败！\n");
        return 1;
    }

    // 文件操作...
```

```
    // 关闭文件（必须！）
    fclose(fp);

    return 0;
}
```

> **一定要检查 `fopen` 的返回值！** 文件可能因权限、路径等原因打开失败。**务必 `fclose`**，否则可能导致数据丢失。

10.4 文件的读写

10.4.1 字符读写：fgetc / fputc

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char ch;

    // 写入字符
    fp = fopen("output.txt", "w");
    if (fp == NULL) return 1;

    fputc('H', fp);
    fputc('i', fp);
    fputc('\n', fp);
    fclose(fp);

    // 读取字符
    fp = fopen("output.txt", "r");
    if (fp == NULL) return 1;

    while ((ch = fgetc(fp)) != EOF)   // EOF: 文件结束标志
    {
        putchar(ch);    // 输出到屏幕
    }
    fclose(fp);

    return 0;
}
```

10.4.2 文件复制案例

```
#include <stdio.h>

int main()
{
    FILE *src, *dest;
```

```
    char ch;

    src = fopen("source.txt", "r");
    if (src == NULL)
    {
        printf("无法打开源文件\n");
        return 1;
    }

    dest = fopen("copy.txt", "w");
    if (dest == NULL)
    {
        printf("无法创建目标文件\n");
        fclose(src);
        return 1;
    }

    // 逐字符复制
    while ((ch = fgetc(src)) != EOF)
        fputc(ch, dest);

    printf("文件复制完成! \n");

    fclose(src);
    fclose(dest);
    return 0;
}
```

10.4.3 字符串读写：fgets / fputs

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char line[256];

    // 写入字符串
    fp = fopen("lines.txt", "w");
    if (fp == NULL) return 1;

    fputs("第一行文字\n", fp);
    fputs("第二行文字\n", fp);
    fputs("第三行文字\n", fp);
    fclose(fp);

    // 读取字符串
    fp = fopen("lines.txt", "r");
    if (fp == NULL) return 1;

    while (fgets(line, sizeof(line), fp) != NULL)
    {
```

```
        printf("%s", line);   // fgets会保留换行符
    }
    fclose(fp);

    return 0;
}
```

> `fgets(buf, size, fp)` 最多读 `size-1` 个字符（留一个给 `'\0'`），遇到换行符或文件尾停止。

10.4.4 格式化读写：fprintf / fscanf（最常用）

```
#include <stdio.h>

typedef struct
{
    char name[20];
    int age;
    float score;
} Student;

// 将学生数据写入文件
void saveStudent(FILE *fp, const Student *s)
{
    fprintf(fp, "%s %d %.1f\n", s->name, s->age, s->score);
}

// 从文件读取学生数据
int loadStudent(FILE *fp, Student *s)
{
    return fscanf(fp, "%s %d %f", s->name, &s->age, &s->score);
    // 成功返回读取的项目数（3），失败或结束返回EOF
}

int main()
{
    FILE *fp;

    // 写入
    fp = fopen("students.txt", "w");
    if (fp == NULL) return 1;

    Student s1 = {"张三", 20, 88.5};
    Student s2 = {"李四", 21, 92.0};
    saveStudent(fp, &s1);
    saveStudent(fp, &s2);
    fclose(fp);

    // 读取
    fp = fopen("students.txt", "r");
    if (fp == NULL) return 1;

    Student s;
```

```
    printf("学号  姓名    年龄  成绩\n");
    int num = 1;
    while (loadStudent(fp, &s) == 3)  // 成功读取3项
    {
        printf("%-5d %-6s %-4d %-5.1f\n", num, s.name, s.age, s.score);
        num++;
    }
    fclose(fp);

    return 0;
}
```

10.4.5 二进制读写：fread / fwrite

```
size_t fread(void *ptr, size_t size, size_t count, FILE *fp);
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *fp);
// size: 每个数据块的大小
// count: 数据块的数量
// 返回实际读写的数据块数
```

```
#include <stdio.h>

typedef struct
{
    int id;
    char name[20];
    float score;
} Student;

int main()
{
    FILE *fp;

    // 二进制写入
    Student s1 = {1001, "张三", 88.5};
    fp = fopen("data.bin", "wb");
    fwrite(&s1, sizeof(Student), 1, fp);
    fclose(fp);

    // 二进制读取
    Student s2;
    fp = fopen("data.bin", "rb");
    fread(&s2, sizeof(Student), 1, fp);
    fclose(fp);

    printf("学号: %d, 姓名: %s, 成绩: %.1f\n", s2.id, s2.name, s2.score);

    return 0;
}
```

> 二进制读写 vs 文本读写：

> - 二进制：速度快、文件小，但人不可读
> - 文本：人可读，调试方便，但速度慢、文件大

10.5 文件状态检测

10.5.1 feof —— 判断文件是否到末尾

```
// 不推荐用法（先读再判断）
while (!feof(fp))
{
    fscanf(fp, "%d", &num);
    printf("%d\n", num);    // 最后一行可能输出两次!
}

// 推荐用法（判断读取函数的返回值）
while (fscanf(fp, "%d", &num) == 1)
{
    printf("%d\n", num);
}
```

10.5.2 ferror —— 检测文件操作错误

```
if (ferror(fp))
{
    printf("文件操作出错! \n");
    clearerr(fp);   // 清除错误标志
}
```

10.6 文件定位（随机读写）

函数	功能
`ftell(fp)`	返回当前位置（距文件开头的字节偏移）
`fseek(fp, offset, origin)`	移动文件位置指针
`rewind(fp)`	回到文件开头（等价于 `fseek(fp, 0, SEEK_SET)`）

fseek 的 origin 参数

常量	含义
`SEEK_SET` (0)	文件开头
`SEEK_CUR` (1)	当前位置
`SEEK_END` (2)	文件末尾

```c
#include <stdio.h>

int main()
{
    FILE *fp;
    char ch;

    // 创建一个测试文件
    fp = fopen("test.txt", "w");
    fprintf(fp, "ABCDEFGHIJKLMNOPQRSTUVWXYZ");
    fclose(fp);

    fp = fopen("test.txt", "r");

    // 从开头偏移5个字节
    fseek(fp, 5, SEEK_SET);
    ch = fgetc(fp);
    printf("第6个字符: %c\n", ch);   // F

    // 从当前位置向后偏移3
    fseek(fp, 3, SEEK_CUR);
    ch = fgetc(fp);
    printf("再向后3个: %c\n", ch);   // J

    // 从末尾向前偏移5
    fseek(fp, -5, SEEK_END);
    ch = fgetc(fp);
    printf("倒数第5个: %c\n", ch);   // V

    // 回到开头
    rewind(fp);
    ch = fgetc(fp);
    printf("第一个: %c\n", ch);       // A

    // 获取文件大小
    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    printf("文件大小: %ld 字节\n", size);  // 26

    fclose(fp);
    return 0;
}
```

10.7 综合案例

案例1：通讯录文件管理系统

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```c
#define MAX 100
#define FILENAME "contacts.txt"

typedef struct
{
    char name[20];
    char phone[20];
    char email[30];
} Contact;

Contact contacts[MAX];
int count = 0;

// 从文件加载
void loadContacts()
{
    FILE *fp = fopen(FILENAME, "r");
    if (fp == NULL) return;

    count = 0;
    while (fscanf(fp, "%s %s %s",
                  contacts[count].name,
                  contacts[count].phone,
                  contacts[count].email) == 3)
    {
        count++;
    }
    fclose(fp);
    printf("已加载 %d 条联系人\n", count);
}

// 保存到文件
void saveContacts()
{
    FILE *fp = fopen(FILENAME, "w");
    if (fp == NULL)
    {
        printf("保存失败！\n");
        return;
    }
    for (int i = 0; i < count; i++)
    {
        fprintf(fp, "%s %s %s\n",
                contacts[i].name,
                contacts[i].phone,
                contacts[i].email);
    }
    fclose(fp);
    printf("已保存 %d 条联系人\n", count);
}

// 添加联系人
```

```c
void addContact()
{
    if (count >= MAX)
    {
        printf("通讯录已满！\n");
        return;
    }
    printf("姓名: "); scanf("%s", contacts[count].name);
    printf("电话: "); scanf("%s", contacts[count].phone);
    printf("邮箱: "); scanf("%s", contacts[count].email);
    count++;
    printf("添加成功！\n");
}

// 查找联系人
void searchContact()
{
    char keyword[20];
    printf("请输入要查找的姓名: ");
    scanf("%s", keyword);

    int found = 0;
    for (int i = 0; i < count; i++)
    {
        if (strstr(contacts[i].name, keyword) != NULL)  // 模糊匹配
        {
            printf("%-15s %-15s %s\n",
                   contacts[i].name,
                   contacts[i].phone,
                   contacts[i].email);
            found = 1;
        }
    }
    if (!found) printf("未找到匹配的联系人\n");
}

// 显示全部
void listAll()
{
    if (count == 0)
    {
        printf("通讯录为空\n");
        return;
    }
    printf("\n===== 全部联系人 (%d人) =====\n", count);
    printf("%-15s %-15s %s\n", "姓名", "电话", "邮箱");
    printf("-----------------------------------------\n");
    for (int i = 0; i < count; i++)
    {
        printf("%-15s %-15s %s\n",
               contacts[i].name,
               contacts[i].phone,
               contacts[i].email);
```

```
    }
}

int main()
{
    loadContacts();

    int choice;
    do
    {
        printf("\n===== 通讯录管理系统 =====\n");
        printf("1. 添加联系人\n");
        printf("2. 查找联系人\n");
        printf("3. 显示全部\n");
        printf("4. 保存并退出\n");
        printf("请选择: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: addContact(); break;
            case 2: searchContact(); break;
            case 3: listAll(); break;
            case 4: saveContacts();
                    printf("再见! \n"); break;
            default: printf("无效选择\n");
        }
    } while (choice != 4);

    return 0;
}
```

案例2：成绩统计分析（从文件读取）

```
// score.txt 内容:
// 张三 85 92 78
// 李四 90 88 95
// 王五 76 82 70

#include <stdio.h>

int main()
{
    FILE *fp = fopen("score.txt", "r");
    if (fp == NULL)
    {
        printf("无法打开score.txt，请先创建该文件\n");
        return 1;
    }

    char name[20];
    int s1, s2, s3;
```

```c
    int count = 0;
    float totalAvg = 0;

    printf("姓名    科目1  科目2  科目3  平均分\n");
    printf("-------------------------------------\n");

    while (fscanf(fp, "%s %d %d %d", name, &s1, &s2, &s3) == 4)
    {
        float avg = (s1 + s2 + s3) / 3.0;
        totalAvg += avg;
        count++;
        printf("%-8s %-6d %-6d %-6d %-6.1f\n", name, s1, s2, s3, avg);
    }

    if (count > 0)
        printf("\n总平均分: %.1f\n", totalAvg / count);

    fclose(fp);
    return 0;
}
```

10.8 本章小结

知识点	要点
文件指针	`FILE *fp;`
打开文件	`fopen("文件名", "模式");` 返回NULL表示失败
关闭文件	`fclose(fp);` 必须关闭！
格式化读写	`fprintf` / `fscanf`（最常用）
字符读写	`fgetc` / `fputc`
字符串读写	`fgets` / `fputs`
二进制读写	`fread` / `fwrite`
文件定位	`fseek` / `ftell` / `rewind`
文件尾判断	`feof(fp)` 或用读取函数返回值

10.9 课后作业

基础练习

1. 选择题

(1) 以只读方式打开名为 `data.txt` 的文件，正确写法是（　）：

A. `fopen("data.txt", "r");`　　B. `fopen("data.txt", "w");`

C. `fopen("data.txt", "a");`　　D. `fopen("data.txt", "r+");`

(2) `fseek(fp, 0, SEEK_END);` 的作用是（　）：

A. 移动到文件开头　　B. 移动到文件末尾

C. 移动到当前位置　　D. 关闭文件

2. 判断题

(1) `fopen` 打开文件后可以不调用 `fclose` 关闭。（　）

(2) 以 `"w"` 模式打开文件不会清空原内容。（　）

编程练习

3. 文件复制

编写程序，将一个文本文件的内容复制到另一个文件。源文件名和目标文件名由用户输入。

4. 统计文件字符数

统计一个文本文件中字母、数字、空格和其他字符的数量。

5. 文件加密

将文本文件中的每个字符进行简单加密（如每个字符的ASCII码+1），保存为新文件。再编写解密程序把加密文件还原。

6. 学生成绩文件读写

从键盘输入10个学生的学号、姓名、三门课成绩，存入文件。然后从文件中读取数据，计算每人总分和平均分，将排名结果写入另一个文件。

7. 日志记录

编写一个简单的日志函数 `void logMsg(const char *msg)`，将带时间戳的消息追加写入 `log.txt` 文件。格式如：`[2025-01-15 10:30:00] 用户登录成功`

> **学习建议**：文件操作让程序具备了"记忆"能力，是实现数据持久化的关键。写完文件后去磁盘上看看文件是否真的生成了，增强感性认识。